

# Actividad 1 ListaLigada

Nombre: Humberto Tijerina Alanis

Materia: Estructura de Datos

GitHub: <https://github.com/FrikiNews/Estructura-de-Datos.git>

Youtube: <https://youtu.be/J4OUyI2PKIk>

## 1. Introducción

Este proyecto consiste en un programa desarrollado en Java, en el entorno Apache NetBeans, que implementa desde cero los tres tipos principales de listas enlazadas: simplemente enlazada, doblemente enlazada y circular. El objetivo es demostrar el manejo de estructuras de datos dinámicas y su capacidad de almacenar distintos tipos de datos gracias al uso de genéricos.

El programa ofrece un menú interactivo por consola donde el usuario puede seleccionar el tipo de lista, insertar, eliminar, buscar y mostrar elementos. Además, incluye una sección de ejemplos con distintos tipos de datos (primitivos, complejos y abstractos) y un pequeño sistema de gestión de contactos que usa las listas como base.

Todas las estructuras fueron programadas manualmente, sin utilizar la clase `LinkedList` de la librería `java.util`, para demostrar la comprensión del funcionamiento interno de estas estructuras.

## 2. Estructura del proyecto

El proyecto está organizado en seis clases, todas dentro del paquete `listaligada`. Cada una cumple una función específica:

| Clase         | Función que cumple                                                                          |
|---------------|---------------------------------------------------------------------------------------------|
| Nodo          | Representa un nodo individual; guarda el dato y las referencias al siguiente y al anterior. |
| ListaSimple   | Lista simplemente enlazada; cada nodo apunta solo al siguiente.                             |
| ListaDoble    | Lista doblemente enlazada; cada nodo apunta al siguiente y al anterior.                     |
| ListaCircular | Lista circular; el último nodo vuelve a apuntar al primero.                                 |
| Contacto      | Objeto con nombre, dirección y teléfono; ejemplo de dato complejo.                          |
| ListaLigada   | Clase principal (Main); contiene el menú y arranca el programa.                             |

## 3. Clase Nodo

En esta sección construí la pieza básica de toda lista enlazada: el nodo. Un nodo almacena un dato y las referencias que lo conectan con otros nodos. Utilicé genéricos (el tipo `<T>`) para que un mismo nodo pueda guardar cualquier tipo de dato: enteros, textos, objetos, etc.

El campo `next` apunta al siguiente nodo, y el campo `prev` apunta al nodo anterior. El campo `prev` solo lo aprovecha la lista doble; las demás listas simplemente lo ignoran.

```
public class Nodo<T> {
    T data;
    Nodo<T> next;
    Nodo<T> prev; // solo lo usa la lista doble

    public Nodo(T data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}
```

```
}  
}
```

## 4. Clase ListaSimple (simplemente enlazada)

Aquí implementé la lista simplemente enlazada. En este tipo de lista cada nodo conoce únicamente al nodo que le sigue. Para hacer más eficientes las inserciones, mantuve dos referencias: head (el primer nodo) y tail (el último nodo), además de un contador size.

### Método insertar

Agrega un nuevo nodo al final de la lista. Si la lista está vacía, el nuevo nodo se convierte en head y tail al mismo tiempo. Si no, se engancha después del tail y ese nuevo nodo pasa a ser el nuevo tail.

### Método eliminar

Busca la primera aparición del dato y lo quita, reconectando el nodo anterior con el siguiente. Se contempla el caso especial de eliminar la cabeza y el de eliminar el último nodo (para actualizar el tail).

### Método buscar

Recorre la lista desde el head comparando cada dato. Devuelve la posición (empezando en 0) donde se encontró el elemento, o -1 si no existe.

### Método mostrar

Recorre la lista de principio a fin e imprime los elementos separados por flechas (->), representando visualmente el enlace entre nodos.

## 5. Clase ListaDoble (doblemente enlazada)

En esta sección desarrollé la lista doblemente enlazada. La diferencia con la simple es que cada nodo guarda una referencia tanto al siguiente como al anterior (prev). Esto permite recorrer la lista en ambos sentidos y hace más sencilla la eliminación de nodos.

### Método insertar

Inserta al final. Al enganchar el nuevo nodo después del tail, también se establece que el prev del nuevo nodo apunte al antiguo tail, manteniendo el doble enlace.

### Método eliminar

Gracias al puntero prev, eliminar es más directo: al encontrar el nodo se reconecta su anterior con su siguiente en ambas direcciones. Se manejan los casos en que el nodo es la cabeza o la cola.

### Método mostrar y mostrarInverso

El método mostrar recorre de head a tail. La ventaja de esta lista es que agregué mostrarInverso, que recorre desde el tail hacia atrás usando el puntero prev, imprimiendo la lista al revés. Esto no se puede hacer fácilmente en la lista simple.

## 6. Clase ListaCircular

Aquí implementé la lista circular. Su característica principal es que el último nodo, en lugar de apuntar a null, apunta de nuevo al primer nodo, formando un círculo. Esto significa que se puede recorrer indefinidamente.

### Método insertar

Cuando se inserta el primer nodo, este se apunta a sí mismo. En las siguientes inserciones, el nuevo nodo se coloca al final y su next se hace apuntar al head para cerrar el círculo, actualizando el tail.

### Método eliminar

Al eliminar hay que tener cuidado de mantener el círculo cerrado. Si se elimina la cabeza, se actualiza head y se reconecta el tail al nuevo head. Se usa un contador de seguridad para no entrar en un bucle infinito al recorrer.

### Método mostrar

Recorre los nodos una sola vuelta (controlada por el contador y el tamaño) e indica al final con un texto que la lista 'vuelve al inicio', dejando claro que es circular.

## 7. Clase Contacto

Esta clase representa el ejemplo de tipo de dato complejo pedido en el proyecto. Un Contacto agrupa tres datos: nombre, dirección y teléfono. Se usa como el elemento que se guarda dentro de las listas en el sistema de gestión de contactos.

Sobrescribí el método equals para que dos contactos se consideren iguales si tienen el mismo nombre (sin importar mayúsculas o minúsculas). Esto permite que las operaciones buscar y eliminar funcionen aunque solo se conozca el nombre. También sobrescribí toString para mostrar el contacto de forma ordenada.

```
public class Contacto {
    private String nombre;
    private String direccion;
    private String telefono;
    // getters, equals (compara por nombre) y toString
}
```

## 8. Clase ListaLigada (programa principal)

Esta es la clase principal que contiene el método main y es el punto de entrada del programa. Aquí construí el menú interactivo que le permite al usuario navegar por todas las funciones.

El menú principal ofrece dos áreas:

- Operar con una lista enlazada: el usuario elige el tipo (simple, doble o circular) y luego puede insertar, eliminar, buscar y mostrar elementos en tiempo real.
- Gestión de contactos: una agenda basada en una lista doble donde se pueden agregar, eliminar, buscar y mostrar contactos.

También incluí un método auxiliar leerEntero que valida la entrada del usuario, evitando que el programa se cierre si se escribe un texto en lugar de un número.

## 9. Resumen de operaciones básicas

Todas las listas comparten el mismo conjunto de operaciones básicas, con el comportamiento que se resume a continuación:

| Operación      | Descripción                                             |
|----------------|---------------------------------------------------------|
| insertar       | Agrega un elemento al final de la lista.                |
| eliminar       | Quita la primera aparición del elemento indicado.       |
| buscar         | Devuelve la posición del elemento o -1 si no existe.    |
| mostrar        | Imprime todos los elementos de la lista en orden.       |
| mostrarInverso | Solo en la lista doble: imprime los elementos al revés. |
| getSize        | Devuelve la cantidad de elementos que tiene la lista.   |

## 10. Conclusión

Con este proyecto implementé de forma completa y funcional los tres tipos de listas enlazadas, cada uno en su propia clase, junto con las operaciones básicas de inserción, eliminación, búsqueda y despliegue. El uso de genéricos permitió que una misma estructura almacenara cualquier tipo de dato, desde primitivos como enteros hasta objetos complejos como los contactos.

El sistema de gestión de contactos mostró una aplicación práctica de estas estructuras en un caso cercano a la vida real. En general, el desarrollo ayudó a comprender a fondo cómo funcionan internamente las listas enlazadas y las diferencias entre cada tipo.